Chess persona modeling

Game preparation, frozen Maia adaptation, and move-conditioned timing

Maia-Chess
21 October 2026

Pipeline and current status

Download player game archives and extract chronological pre-move examples.

Train separate player adapters on a frozen Maia-3 5M backbone.

Predict legal moves and move-conditioned time distributions.

The repository contains a runnable pipeline, not validated grandmaster personas.

Joint prediction task

$$P(\text{move},\text{time} \mid x,p) = P(\text{move} \mid x,p)\, P(\text{time} \mid \text{move},x,p)$$

Factor the joint distribution into move choice and time conditional on the move.

Condition both on board and historical context.

Train one independent checkpoint per player.

Retain a pre-move marginal timing prediction for honest evaluation.

Archive acquisition and caching

Fetch Chess.com monthly archives sequentially.

Cache completed months and refresh the current month.

Limit by date or number of months for an initial coverage audit.

Downloaded games and model checkpoints stay outside tracked source files.

PGN preparation and exclusions

Extract eight-position board history and the last 16 plies.

Remove duplicates and audit coverage by player and time class.

Exclude unsupported variants, staged controls, correspondence games, and unknown dates.

Keep unavailable timing labels masked rather than inventing durations.

Chronological game-level splits

Split entire games chronologically into train, validation, and test.

Keep both sides of a game together, including games between target players.

Keep equal timestamps in the same split.

Filter to the target player before training or sampling.

Clock-consumption labels

duration = clock_before + increment − clock_after

For a single-stage Fischer increment, infer elapsed clock consumption from adjacent same-side clocks.

The first move by each side has no timing label.

A missing clock breaks the next same-side subtraction.

Mask negative or over-budget durations.

Relative-duration bins

relative duration = duration / clock_before

Normalize duration by the clock available before the move.

Use 32 bins from near-instant play to the remaining budget.

The first interval is [0, 0.0001), followed by logarithmic intervals up to one.

Bin centers approximate means and medians in seconds.

Frozen Maia representations

Load the pinned Maia-3 5M implementation and compatible checkpoint.

Extract 64 square representations and legal candidate logits.

Keep backbone parameters frozen.

Use the upstream move vocabulary and tokenization, including promotions.

History and context encoding

Encode source and destination squares, promotion, side, past duration, and missingness.

A 64-dimensional GRU summarizes the last 16 plies.

Fuse pooled board features with 15 numeric context features and a 32-dimensional player embedding.

Project the combined context to 128 dimensions.

Player adapter and legal-move scorer

P(move | x,p) = softmax(base logits + learned correction)

Use a residual adapter with dimensions 128, 32, and 128.

Combine adapted context with each legal move’s square and promotion features.

Predict a scalar correction to its base policy logit.

Initialize the policy correction to zero.

Move-conditioned timing head

Project each candidate representation through 128, 64, and 32 output dimensions.

Softmax produces a distribution over relative-duration bins.

Training conditions timing loss on the observed move when a valid label exists.

Policy learning can use positions without timing labels.

Training objective

$L = CE_move + 0.5\ CE_time + 0.02\ KL(base \Vert adapted)$

Use legal-move cross-entropy and masked conditional timing cross-entropy.

Regularize the adapted policy toward the frozen base distribution.

Default weights are 0.5 for timing and 0.02 for KL.

AdamW uses learning rate 0.0003 and gradient clipping 1.0.

Budgeted training and checkpoints

An explicit update budget overrides epoch count.

Validate at fixed successful-update intervals and at the final update.

best.pt uses combined move and marginal-timing validation score.

best-move.pt uses move NLL and last.pt preserves the completed budget.

Pre-move marginal timing

$P(\text{time} \mid x,p) = \Sigma_{\text{move}} P(\text{move} \mid x,p)\, P(\text{time} \mid \text{move},x,p)$

Weight every move-conditioned timing distribution by its predicted move probability.

Evaluate marginal timing NLL before the actual move is known.

Report conditional timing NLL separately.

Do not compare raw training objective directly with the different validation score.

Metrics and empirical baselines

Report move accuracy and NLL against the frozen backbone.

Report conditional and marginal timing NLL, approximate MAE, and interval coverage and width.

Group by time class, exact base+increment, and remaining-clock band.

Fit histogram and median-time baselines on training data only.

Execution and saved artifacts

download and prepare create audited processed splits.

train --player NAME produces the independent player checkpoint.

evaluate reads a checkpoint and writes test metrics.

Coverage, source hashes, selected offsets, batch-order hashes, and timing baselines support reproducibility.

Generalization and interpretation

Synthetic tests verify software behavior, not style fidelity or playing strength.

Different time controls and eras confound player-speed comparisons.

Maia pretraining may overlap older downstream evaluation games.

Reserve suitably recent external games for final confirmation.

Source map

Repository snapshot

Maia-Chess
1bc429b5517280819910fb573b0593a770f0a904

The notes on each slide link to the code and reports supporting its explanation. Reported experiment outcomes retain the scope and limitations stated in those sources.

21 October 2026